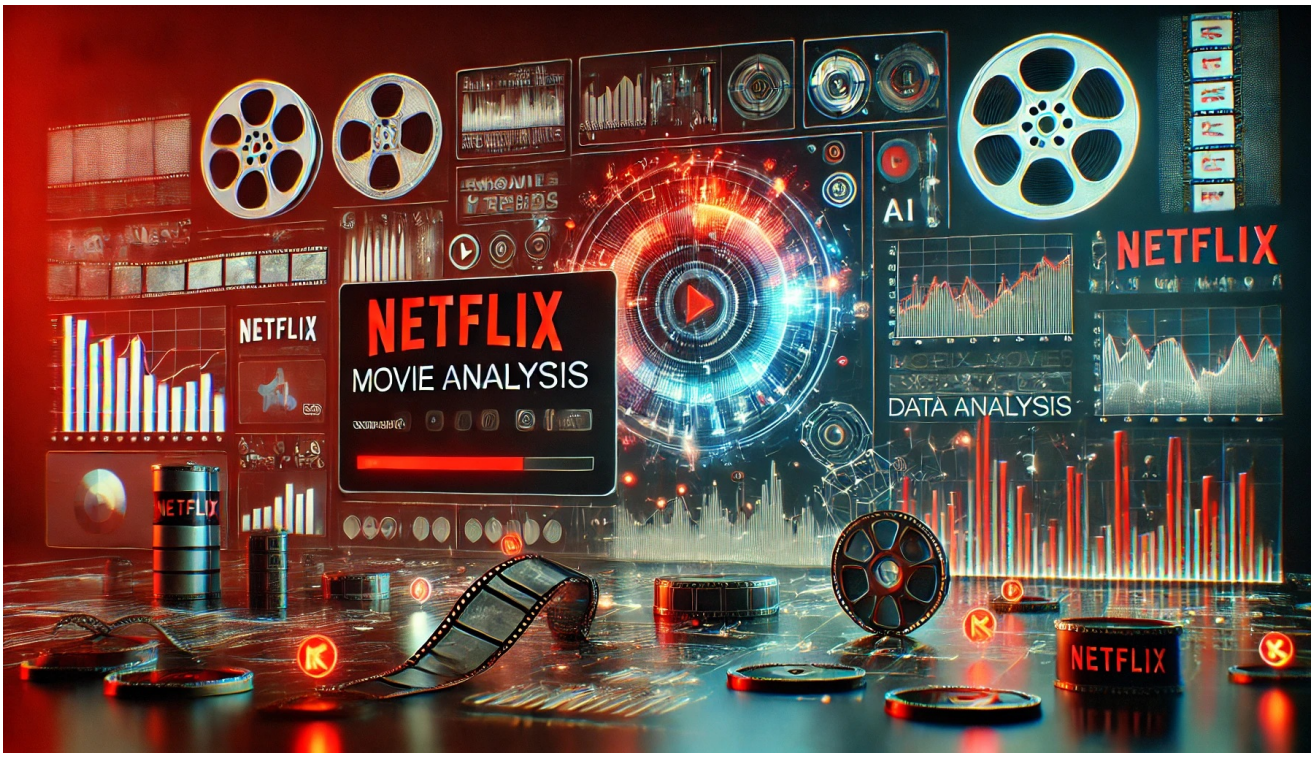


Netflix Movie Data Analysis Problem Statement:-

Netflix leverages data science, AI, and machine learning to enhance its recommendation system and understand customer behavior. Suppose you are working in a data-driven role with a dataset of 9,000+ movies. To help the company make informed business decisions, you need to answer the following:-

1)General Insights

- What is the distribution of movie popularity?
- What is the distribution of vote count and vote average?
- How does popularity correlate with vote count and vote average?
- Which are the most popular and highest-rated movies?

2)Genre Analysis

- What are the top genres based on the number of movies?
- Which genres have the highest popularity on average?
- Do certain genres receive more votes or higher ratings?

3)Release Date & Trends

- How many movies are released each year?
- What are the trends in movie ratings over time?
- Does popularity or vote count change over the years?

4)Language Analysis

- What are the most common languages in the dataset?
- How do movies in different languages compare in terms of ratings and popularity?

5)Relationship Between Popularity & Ratings

- Is there a correlation between popularity and ratings?
- Do highly rated movies tend to be more or less popular?

Essential Steps in Exploratory Data Analysis (EDA)

1. Importing Libraries – Load essential Python libraries such as pandas, numpy, seaborn, and matplotlib.
2. Loading the Dataset – Read the dataset into a dataframe for analysis.
3. Handling Missing Values – Identify and manage null or missing values.
4. Exploring Numerical Variables – Analyze statistical properties, distributions, and outliers.
5. Exploring Categorical Variables – Examine unique values, frequencies, and distributions.
6. Feature Relationships & Insights – Use correlation, pair plots, and visualizations to understand feature dependencies.

Importing Some Important Library.

In [2]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')
```

Loading and reading the dataset

In [3]:
```
df=pd.read_csv('mymoviedb.csv',lineterminator='\n')
df
```

Out[3]:

	Release_Date	Title	Overview	Popularity	Vote_Count	Vote_Average	Original_Language	Genre	
0	2021-12-15	Spider-Man: No Way Home	Peter Parker is unmasked and no longer able to...	5083.954	8940	8.3	en	Action, Adventure, Science Fiction	https://image.tmdb.org/t/p/c
1	2022-03-01	The Batman	In his second year of fighting crime, Batman u...	3827.658	1151	8.1	en	Crime, Mystery, Thriller	https://image.tmdb.org/t/p/or
2	2022-02-25	No Exit	Stranded at a rest stop in the mountains durin...	2618.087	122	6.3	en	Thriller	https://image.tmdb.org/t/p/orig
3	2021-11-24	Encanto	The tale of an extraordinary family, the Madri...	2402.201	5076	7.7	en	Animation, Comedy, Family, Fantasy	https://image.tmdb.org/t/p/or
4	2021-12-22	The King's Man	As a collection of history's worst tyrants and...	1895.511	1793	7.0	en	Action, Adventure, Thriller, War	https://image.tmdb.org/t/p/ori
...	...	...	...	...	...	...	...	...	
9822	1973-10-15	Badlands	A dramatization of the Starkweather-Fugate kil...	13.357	896	7.6	en	Drama, Crime	https://image.tmdb.org/t/p/o
9823	2020-10-01	Violent Delights	A female vampire falls in love with a man she ...	13.356	8	3.5	es	Horror	https://image.tmdb.org/t/p/or
9824	2016-05-06	The Offering	When young and successful reporter Jamie finds...	13.355	94	5.0	en	Mystery, Thriller, Horror	https://image.tmdb.org/t/p/orig
9825	2021-03-31	The United States vs. Billie Holiday	Billie Holiday spent much of her career being ...	13.354	152	6.7	en	Music, Drama, History	https://image.tmdb.org/t/p/o
9826	1984-09-23	Threads	Documentary style account of a nuclear holocau...	13.354	186	7.8	en	War, Drama, Science Fiction	https://image.tmdb.org/t/p/ori

9827 rows × 9 columns

Top five rows from the start

Top five rows from the start.

In [4]:
```
df.head()
```

Out[4]:

	Release_Date	Title	Overview	Popularity	Vote_Count	Vote_Average	Original_Language	Genre	
0	2021-12-15	Spider-Man: No Way Home	Peter Parker is unmasked and no longer able to...	5083.954	8940	8.3	en	Action, Adventure, Science Fiction	https://image.tmdb.org/t/p/original
1	2022-03-01	The Batman	In his second year of fighting crime, Batman u...	3827.658	1151	8.1	en	Crime, Mystery, Thriller	https://image.tmdb.org/t/p/original/
2	2022-02-25	No Exit	Stranded at a rest stop in the mountains durin...	2618.087	122	6.3	en	Thriller	https://image.tmdb.org/t/p/original/v
3	2021-11-24	Encanto	The tale of an extraordinary family, the Madri...	2402.201	5076	7.7	en	Animation, Comedy, Family, Fantasy	https://image.tmdb.org/t/p/original/
4	2021-12-22	The King's Man	As a collection of history's worst tyrants and...	1895.511	1793	7.0	en	Action, Adventure, Thriller, War	https://image.tmdb.org/t/p/original/a

Top five rows from the bottom/end

In [5]:
```
df.tail()
```

Out[5]:

	Release_Date	Title	Overview	Popularity	Vote_Count	Vote_Average	Original_Language	Genre	
9822	1973-10-15	Badlands	A dramatization of the Starkweather-Fugate kil...	13.357	896	7.6	en	Drama, Crime	https://image.tmdb.org/t/p/orig
9823	2020-10-01	Violent Delights	A female vampire falls in love with a man she ...	13.356	8	3.5	es	Horror	https://image.tmdb.org/t/p/origi
9824	2016-05-06	The Offering	When young and successful reporter Jamie finds...	13.355	94	5.0	en	Mystery, Thriller, Horror	https://image.tmdb.org/t/p/origina
9825	2021-03-31	The United States vs. Billie Holiday	Billie Holiday spent much of her career being ...	13.354	152	6.7	en	Music, Drama, History	https://image.tmdb.org/t/p/origi
9826	1984-09-23	Threads	Documentary style account of a nuclear holocau...	13.354	186	7.8	en	War, Drama, Science Fiction	https://image.tmdb.org/t/p/origi

Check how many rows and columns are present in this dataset.

In [6]:
```
print("The Number of rows",df.shape[0])
print("The Number of columns",df.shape[1])
```

```
The Number of rows 9827
The Number of columns 9
```

Observation:-

- There are 9827 rows and 9 columns

Check Information of the Dataset

In [7]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9827 entries, 0 to 9826
Data columns (total 9 columns):
 #   Column             Non-Null Count  Dtype  
---  ------             --------------  -----  
 0   Release_Date       9827 non-null   object 
 1   Title              9827 non-null   object 
 2   Overview           9827 non-null   object 
 3   Popularity         9827 non-null   float64
 4   Vote_Count         9827 non-null   int64  
 5   Vote_Average       9827 non-null   float64
 6   Original_Language  9827 non-null   object 
 7   Genre              9827 non-null   object 
 8   Poster_Url         9827 non-null   object 
dtypes: float64(2), int64(1), object(6)
memory usage: 691.1+ KB
```

Check the numbers of null value present in this dataset

In [8]:
```
df.isnull().sum()
```

Out[8]:
```
Release_Date         0
Title                0
Overview             0
Popularity           0
Vote_Count           0
Vote_Average         0
Original_Language    0
Genre                0
Poster_Url           0
dtype: int64
```

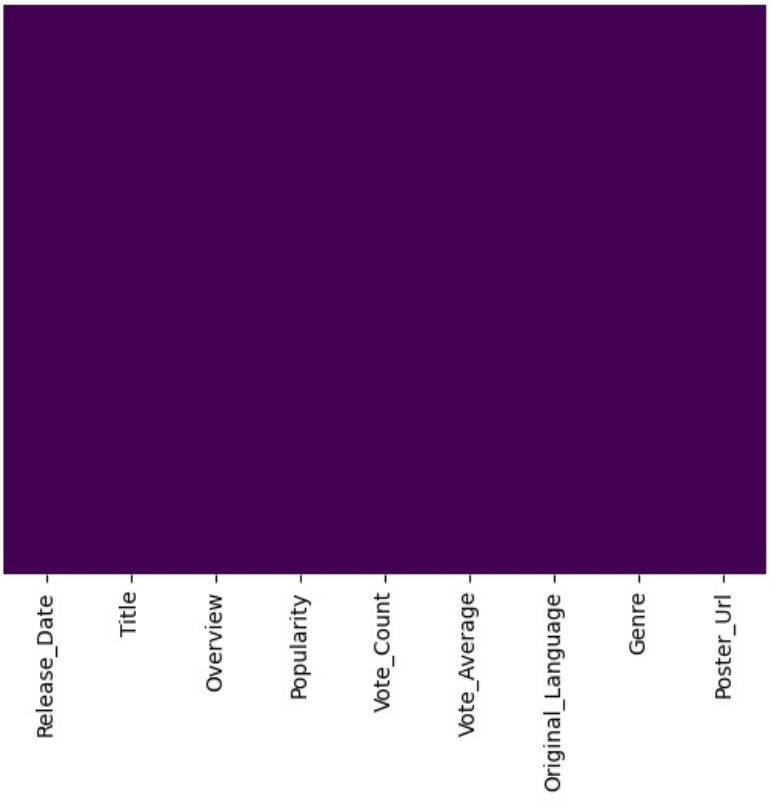
Observation:-

There are no any null-value present of this datasets.

visualize these null value by using the heatmap

In [9]:
```
sns.heatmap(df.isnull(),yticklabels=False,cbar=False,cmap='viridis')
```

Out[9]:
```
<Axes: >
```



Perform Statistical analysis

In [10]:
```
jac=df.describe()
jac
```

Out[10]:

	Popularity	Vote_Count	Vote_Average
count	9827.000000	9827.000000	9827.000000
mean	40.326088	1392.805536	6.439534
std	108.873998	2611.206907	1.129759
min	13.354000	0.000000	0.000000
25%	16.128500	146.000000	5.900000
50%	21.199000	444.000000	6.500000
75%	35.191500	1376.000000	7.100000
max	5083.954000	31077.000000	10.000000

Observation:-

Popularity

- Highly skewed, with a few movies having extreme popularity (max: 5083.95).
- Most movies have low popularity (median: 21.20).

Vote Count

- Large variation (std: 2611.21), with some movies having zero votes while others exceed 31,000 votes.
- Most movies have relatively low votes (median: 444).

Vote Average (Rating)

- Most movies are moderately rated (5.9 - 7.1).
- Some movies have a perfect 10, while others have 0 ratings.

Check the number of unique values in each column.

In [11]:
```
vil=df.nunique()
vil
```

Out[11]:
```
Release_Date         5893
Title                9513
Overview             9822
Popularity           8160
Vote_Count           3266
Vote_Average           74
Original_Language      43
Genre                2337
Poster_Url           9827
dtype: int64
```

In [12]:
```
df.columns
```

Out[12]:
```
Index(['Release_Date', 'Title', 'Overview', 'Popularity', 'Vote_Count',
       'Vote_Average', 'Original_Language', 'Genre', 'Poster_Url'],
      dtype='object')
```

Categorical feature

In [13]:
```
categorical_features = [feature for feature in df.columns if df[feature].dtype=='O']
categorical_features
```

Out[13]:
```
['Release_Date',
 'Title',
 'Overview',
 'Original_Language',
 'Genre',
 'Poster_Url']
```

Number of Categorical feature

In [14]:
```
print(len([feature for feature in df.columns if df[feature].dtype=='O']))
```

```
6
```

Observation:-

- There are 6 categorical feature present of this datasets

Numerical columns

In [15]:
```
numerical_features = [feature for feature in df.columns if df[feature].dtype!='O']
numerical_features
```

Out[15]:
```
['Popularity', 'Vote_Count', 'Vote_Average']
```

Number of Numerical feature

In [16]:
```
print(len([feature for feature in df.columns if df[feature].dtype!='O']))
```

```
3
```

Observation:-

- There are 3 numerical feature present of this datasets

1)General Insights:-

- What is the distribution of movie popularity?
- What is the distribution of vote count and vote average?
- How does popularity correlate with vote count and vote average?
- Which are the most popular and highest-rated movies?

What is the distribution of movie popularity

In [17]:
```
# Calculate descriptive statistics for the popularity distribution
popularity_stats = df['Popularity'].describe()
print("Descriptive Statistics for Movie Popularity:")
print(popularity_stats)
```

```
Descriptive Statistics for Movie Popularity:
count    9827.000000
mean       40.326088
std       108.873998
min        13.354000
25%        16.128500
50%        21.199000
75%        35.191500
max      5083.954000
Name: Popularity, dtype: float64
```

Observations:-

- Popularity is highly skewed, with a few movies being extremely popular (max: 5083.95).
- Most movies have low popularity (median: 21.20, 75% below 35.19).
- The high standard deviation (108.87) indicates large variability in popularity

In [18]:
```
# Show percentiles for better understanding of the distribution
percentiles = [0.1, 1, 5, 10, 25, 50, 75, 90, 95, 99, 99.9]
print("Percentiles of Popularity Distribution:-")
for p in percentiles:
    value = np.percentile(df['Popularity'].dropna(), p)
    print(f"{p}th percentile: {value:.2f}")
```

```
Percentiles of Popularity Distribution:-
0.1th percentile: 13.36
1th percentile: 13.43
5th percentile: 13.80
10th percentile: 14.29
25th percentile: 16.13
50th percentile: 21.20
75th percentile: 35.19
90th percentile: 66.21
95th percentile: 106.53
99th percentile: 282.20
99.9th percentile: 1506.94
```

Observations:-

- Most movies have very low popularity (below 21.20 for 50% of movies).
- 90% of movies have popularity below 66.21, showing a concentrated low range.
- Only 1% of movies exceed 282.20, and 0.1% exceed 1506.94, confirming extreme skewness.
- A few movies dominate popularity, while most remain low-profile.

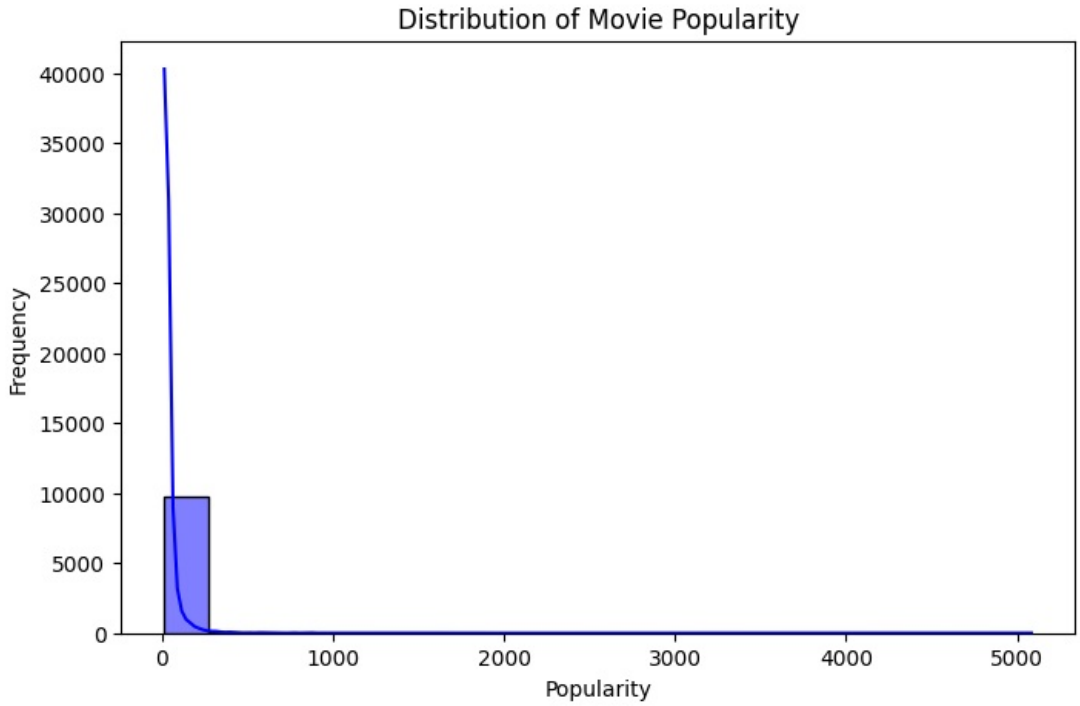
In [19]:
```python
# Count movies in different popularity ranges
print("Popularity Range Distribution:")
ranges = [(0, 100), (100, 500), (500, 1000), (1000, 2000), (2000, 5000), (5000, float('inf'))]
for low, high in ranges:
    count = df[(df['Popularity'] >= low) & (df['Popularity'] < high)].shape[0]
    percentage = (count / df.shape[0]) * 100
    print(f"Movies with popularity {low}-{high}: {count} ({percentage:.2f}%)")
```

```
Popularity Range Distribution:
Movies with popularity 0-100: 9278 (94.41%)
Movies with popularity 100-500: 500 (5.09%)
Movies with popularity 500-1000: 26 (0.26%)
Movies with popularity 1000-2000: 19 (0.19%)
Movies with popularity 2000-5000: 3 (0.03%)
Movies with popularity 5000-inf: 1 (0.01%)
```

Observations:-

- 94.41% of movies have low popularity (below 100), confirming that most movies are not widely popular.
- Only 5.09% of movies have moderate popularity (100-500).
- Less than 1% of movies exceed 500 popularity, showing extreme rarity.
- Only 1 movie (0.01%) has over 5000 popularity, highlighting a massive outlier.

In [20]:
```python
# Visualization distribution of movie popularity
plt.figure(figsize=(8, 5))
sns.histplot(df['Popularity'].dropna(), kde=True, bins=20, color='blue')
plt.title('Distribution of Movie Popularity')
plt.xlabel('Popularity')
plt.ylabel('Frequency')
plt.show()
```



Observation:-

The distribution of movie popularity is heavily right-skewed, with most movies having relatively low popularity scores and only a few movies having very high popularity.

What is the distribution of vote count and vote average?

In [21]:
```
# Calculate descriptive statistics for vote count
vote_count_stats = df['Vote_Count'].describe()
print("Descriptive Statistics for Vote Count:-")
print(vote_count_stats)
```

```
Descriptive Statistics for Vote Count:-
count     9827.000000
mean      1392.805536
std       2611.206907
min          0.000000
25%        146.000000
50%        444.000000
75%       1376.000000
max      31077.000000
Name: Vote_Count, dtype: float64
```

Observations:-

- Highly skewed distribution with few movies getting massive votes (max: 31,077).
- 50% of movies have fewer than 444 votes, showing most movies get low engagement.
- 75% of movies have under 1,376 votes, meaning only a few movies receive thousands.
- Some movies have 0 votes, indicating they are unrated or unpopular.

In [22]:
```
# Calculate additional metrics for vote count
print("Additional Distribution Metrics for Vote Count:-")
print("Skewness:", df['Vote_Count'].skew())
print("Kurtosis:", df['Vote_Count'].kurtosis())
```

```
Additional Distribution Metrics for Vote Count:-
Skewness: 4.1035928883875314
Kurtosis: 22.309870929469398
```

Observations on Vote Count Distribution:-

- High skewness (4.10) indicates a strong right skew, meaning most movies have low votes, while a few have extremely high votes.
- Very high kurtosis (22.31) suggests a heavy-tailed distribution, meaning there are many extreme outliers (movies with exceptionally high votes).

In [23]:
```
# Percentiles for vote count distribution
percentiles = [0.1, 1, 5, 10, 25, 50, 75, 90, 95, 99, 99.9]
print("Percentiles of Vote Count Distribution:-")
for p in percentiles:
    value = np.percentile(df['Vote_Count'].dropna(), p)
    print(str(p) + "th percentile:", round(value, 2))
```

```
Percentiles of Vote Count Distribution:-
0.1th percentile: 0.0
1th percentile: 0.0
5th percentile: 15.0
10th percentile: 47.0
25th percentile: 146.0
50th percentile: 444.0
75th percentile: 1376.0
90th percentile: 3647.0
95th percentile: 6064.4
99th percentile: 13870.04
99.9th percentile: 22404.25
```

Observations:-

- 1% of movies have 0 votes, meaning some movies are completely unrated.
- 50% of movies have ≤ 444 votes, showing that most movies receive low engagement.
- Only 10% of movies exceed 3,647 votes, confirming a highly skewed distribution.
- 99.9th percentile (22,404 votes) indicates a few blockbuster movies dominate vote counts.

In [24]:
```
# Count movies in different vote count ranges
```

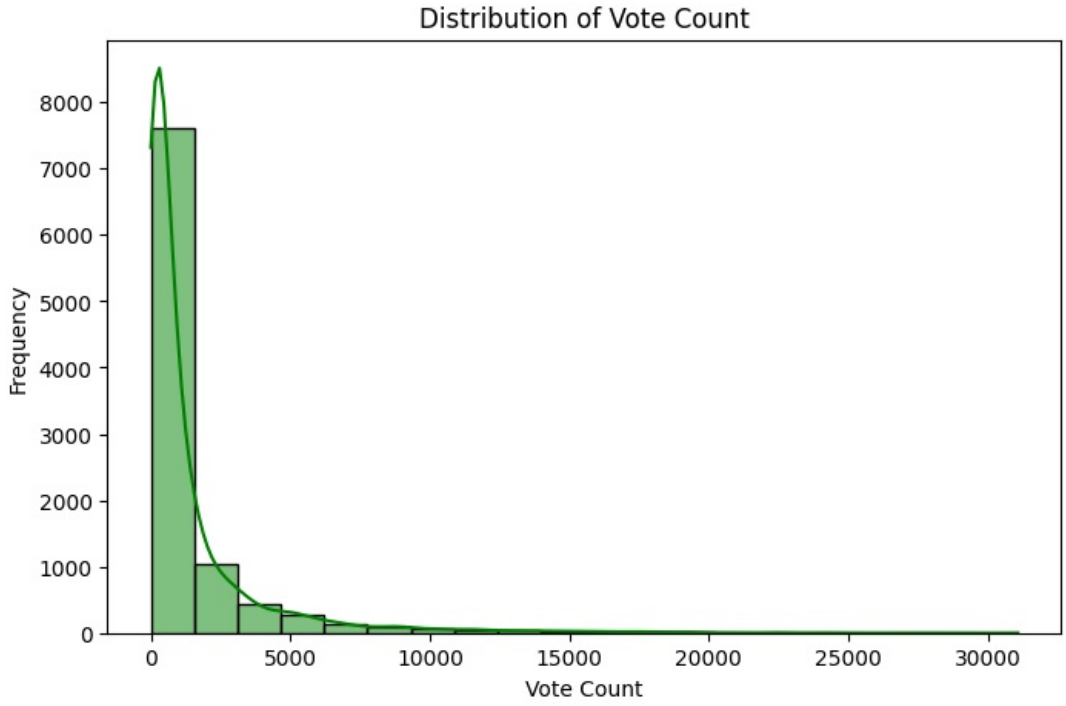
```
print("Vote Count Range Distribution:-")
vote_count_ranges = [(0, 100), (100, 500), (500, 1000), (1000, 2000), (2000, 5000), (5000, float('inf'))]
for low, high in vote_count_ranges:
    count = df[(df['Vote_Count'] >= low) & (df['Vote_Count'] < high)].shape[0]
    percentage = (count / df.shape[0]) * 100
    print("Movies with vote count " + str(low) + "-" + str(high) + ": " + str(count) + " (" + str(round(percent
```

```
Vote Count Range Distribution:-
Movies with vote count 0-100: 1773 (18.04%)
Movies with vote count 100-500: 3406 (34.66%)
Movies with vote count 500-1000: 1490 (15.16%)
Movies with vote count 1000-2000: 1363 (13.87%)
Movies with vote count 2000-5000: 1117 (11.37%)
Movies with vote count 5000-inf: 678 (6.9%)
```

Observations:-

- 34.66% of movies have 100-500 votes, making it the most common range.
- 18.04% of movies have less than 100 votes, indicating low engagement.
- Only 6.9% of movies exceed 5,000 votes, showing a small number of highly rated films.
- The distribution is skewed, with few movies dominating vote counts.

In [25]:
```
# Visualization Distribution of vote count
plt.figure(figsize=(8, 5))
sns.histplot(df['Vote_Count'].dropna(), kde=True, bins=20, color='green')
plt.title('Distribution of Vote Count')
plt.xlabel('Vote Count')
plt.ylabel('Frequency')
plt.show()
```



Observation :-

Similar to popularity, the vote count distribution is also right-skewed, indicating that most movies receive relatively few votes while a small number of movies receive a large number of votes.

In [26]:
```
# Calculate descriptive statistics for vote average
vote_avg_stats = df['Vote_Average'].describe()
print("Descriptive Statistics for Vote Average:-")
print(vote_avg_stats)
```

```
Descriptive Statistics for Vote Average:-
count    9827.000000
mean        6.439534
std         1.129759
min         0.000000
25%         5.900000
50%         6.500000
75%         7.100000
max        10.000000
Name: Vote_Average, dtype: float64
```

Observations:-

- Most movies have ratings between 5.9 and 7.1 (25th-75th percentile).
- Median rating is 6.5, meaning typical movies are rated moderately well.
- Some movies have a perfect 10, while a few have 0 ratings.
- Low standard deviation (1.13) suggests ratings are not highly spread out.

In [27]:
```
# Calculate additional metrics for vote average
print("Additional Distribution Metrics for Vote Average:-")
print("Skewness", df['Vote_Average'].skew())
print("Kurtosis", df['Vote_Average'].kurtosis())
```

```
Additional Distribution Metrics for Vote Average:-
Skewness -2.085441859272996
Kurtosis 9.592512204404132
```

Observations:-

- Negative skewness (-2.08) indicates a left-skewed distribution, meaning more movies have higher ratings than lower ones.
- High kurtosis (9.59) suggests a peaked distribution with extreme outliers, meaning some movies have exceptionally low or high ratings.

In [28]:
```
# Percentiles for vote average distribution
print("Percentiles of Vote Average Distribution:-")
for p in percentiles:
    value = np.percentile(df['Vote_Average'].dropna(), p)
    print(str(p) + "th percentile:", round(value, 2))
```

```
Percentiles of Vote Average Distribution:-
0.1th percentile: 0.0
1th percentile: 0.0
5th percentile: 4.8
10th percentile: 5.3
25th percentile: 5.9
50th percentile: 6.5
75th percentile: 7.1
90th percentile: 7.6
95th percentile: 7.9
99th percentile: 8.4
99.9th percentile: 8.9
```

Observations:-

- 1% of movies have a rating of 0, indicating unrated or very poorly rated movies.
- 50% of movies are rated ≤ 6.5, showing most movies have moderate ratings.
- Only 1% of movies exceed 8.4, meaning very high-rated movies are rare.
- Most movies (75%) have ratings between 5.9 and 7.1, confirming a tight rating distribution.

In [29]:
```
# Count movies in different vote average ranges
print("Vote Average Range Distribution:-")
vote_avg_ranges = [(0, 4), (4, 5), (5, 6), (6, 7), (7, 8), (8, 9), (9, 10)]
for low, high in vote_avg_ranges:
    count = df[(df['Vote_Average'] >= low) & (df['Vote_Average'] < high)].shape[0]
    percentage = (count / df.shape[0]) * 100
    print("Movies with vote average " + str(low) + "-" + str(high) + ": " + str(count) + " (" + str(round(perce
```

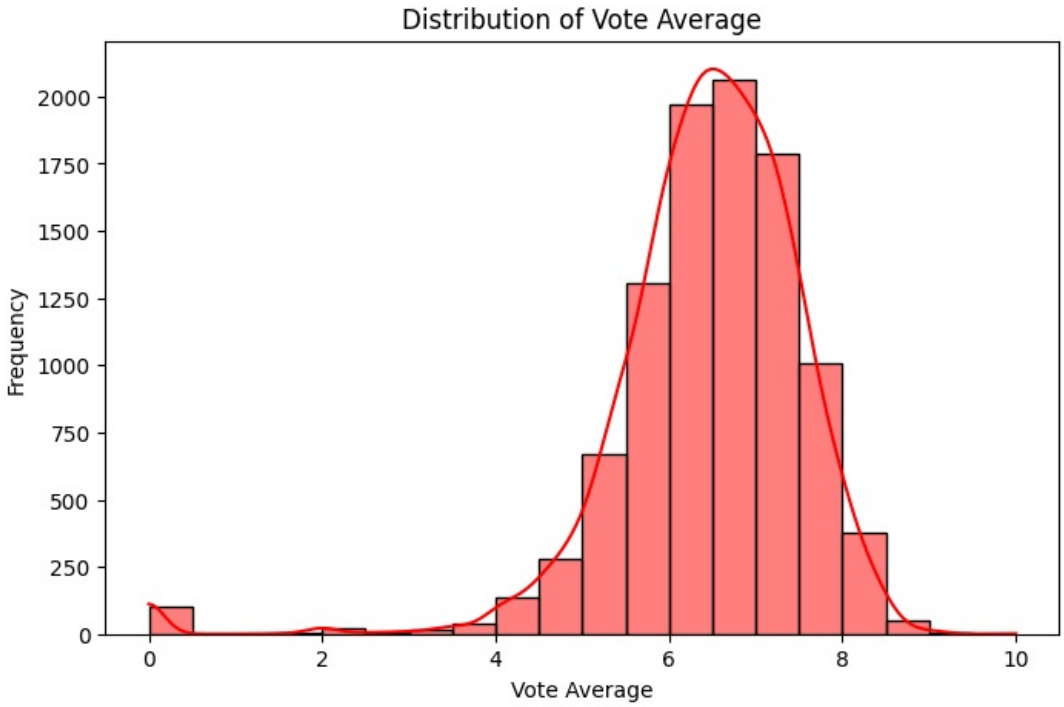
```
Vote Average Range Distribution:-
Movies with vote average 0-4: 179 (1.82%)
Movies with vote average 4-5: 414 (4.21%)
Movies with vote average 5-6: 1974 (20.09%)
Movies with vote average 6-7: 4033 (41.04%)
Movies with vote average 7-8: 2791 (28.4%)
Movies with vote average 8-9: 428 (4.36%)
Movies with vote average 9-10: 7 (0.07%)
```

Observations:-

- Most movies (41.04%) have ratings between 6-7, making it the most common range.
- 28.4% of movies have good ratings (7-8), showing many well-rated films.
- Only 0.07% of movies have a rating of 9-10, making highly rated movies extremely rare.
- Very few movies (1.82%) are rated below 4, indicating that most movies receive average or better ratings.

In [30]:
```
# visuaization Distribution of vote average
plt.figure(figsize=(8, 5))
sns.histplot(df['Vote_Average'].dropna(), kde=True, bins=20, color='red')
```

```
plt.title('Distribution of Vote Average')
plt.xlabel('Vote Average')
plt.ylabel('Frequency')
plt.show()
```



Observation:-

The vote average distribution appears more normally distributed, centered around 6-7, suggesting that most movies receive average ratings.

How does popularity correlate with vote count and vote average?

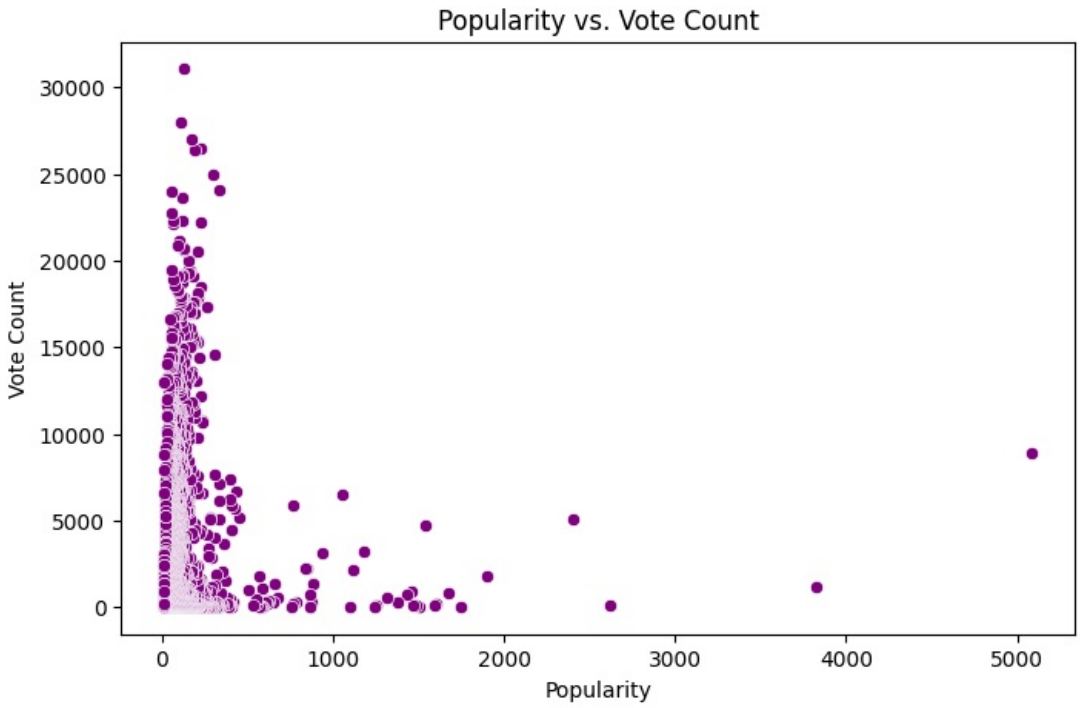
In [31]:
```
# Correlation matrix
corr_matrix = df[['Popularity', 'Vote_Count', 'Vote_Average']].corr()
print('Correlation Matrix:')
print(corr_matrix)
```

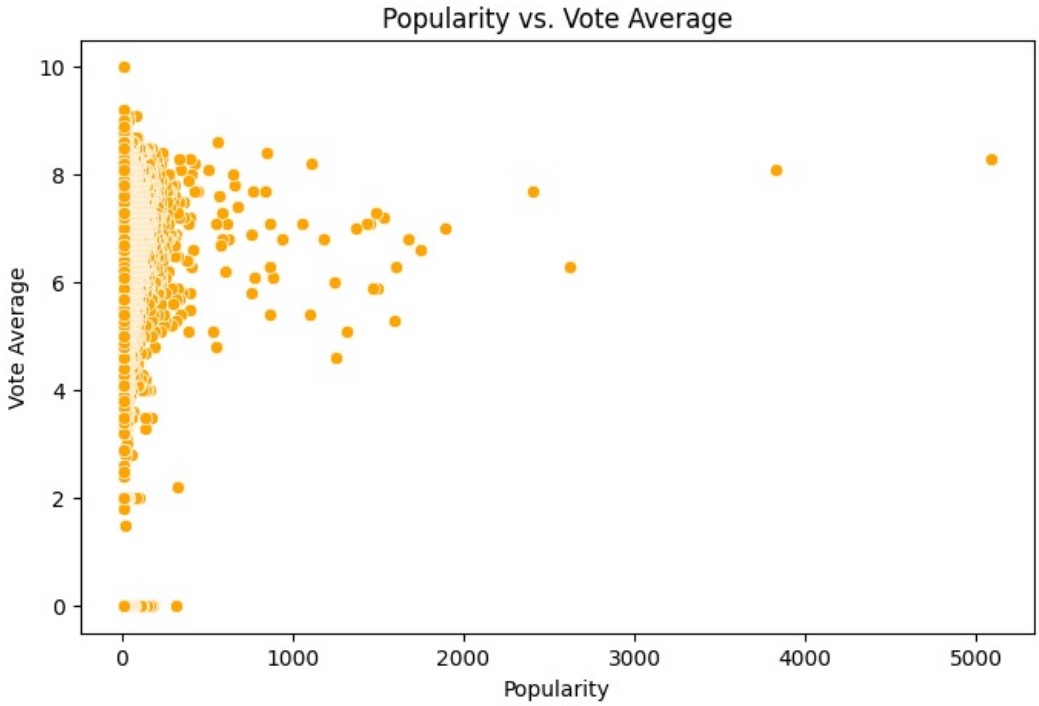
```
Correlation Matrix:
              Popularity  Vote_Count  Vote_Average
Popularity      1.000000    0.137400      0.053844
Vote_Count      0.137400    1.000000      0.253574
Vote_Average    0.053844    0.253574      1.000000
```

Observations:-

- Popularity & Vote Count (0.137) → Weak positive correlation; popular movies tend to get more votes, but not strongly.
- Vote Count & Vote Average (0.254) → Weak positive correlation; more votes slightly indicate higher ratings.
- Popularity & Vote Average (0.054) → Almost no correlation, meaning a movie's rating doesn't strongly affect its popularity.

In [32]:
```
# visualization of Popularity vs Vote Count and also Popularity vs Vote Average
plt.figure(figsize=(8, 5))
sns.scatterplot(x='Popularity', y='Vote_Count', data=df, color='purple')
plt.title('Popularity vs. Vote Count')
plt.xlabel('Popularity')
plt.ylabel('Vote Count')
plt.show()

plt.figure(figsize=(8, 5))
sns.scatterplot(x='Popularity', y='Vote_Average', data=df, color='orange')
plt.title('Popularity vs. Vote Average')
plt.xlabel('Popularity')
plt.ylabel('Vote Average')
plt.show()
```





Observation

The correlation analysis shows:-

- A weak positive correlation (0.137) between popularity and vote count
- A very weak positive correlation (0.054) between popularity and vote average
- A moderate positive correlation (0.254) between vote count and vote average

Which are the most popular and highest-rated movies?

In [33]:
```
# Most popular movies
most_popular = df.sort_values(by='Popularity', ascending=False).head(5)
print('Most Popular Movies:')
print(most_popular[['Title', 'Popularity']])
```

```
Most Popular Movies:
                     Title  Popularity
0  Spider-Man: No Way Home    5083.954
1               The Batman    3827.658
2                  No Exit    2618.087
3                  Encanto    2402.201
4           The King's Man    1895.511
```

Observation:-

Spider-Man: No Way Home is by far the most popular movie in the dataset, followed by The Batman and No Exit.

In [34]:
```
# Highest-rated movies (by Vote_Average)
highest_rated = df.sort_values(by='Vote_Average', ascending=False).head(5)
print('Highest-Rated Movies:')
print(highest_rated[['Title', 'Vote_Average']])
```

```
Highest-Rated Movies:
                                          Title  Vote_Average
9391                    Kung Fu Master Huo Yuanjia          10.0
7339             Franco Escamilla: Por La Anécdota           9.2
2325                              Impossible Things           9.1
667   Demon Slayer: Kimetsu no Yaiba Sibling's Bond           9.1
7014                    Sex School: Dorms of Desire           9.0
```

Observation:-

Interestingly, the highest-rated movies are not necessarily the most popular ones. "Kung Fu Master Huo Yuanjia" has a perfect 10.0 rating, followed by "Franco Escamilla: Por La Anécdota" with 9.2.

2)Genre Analysis:-

- What are the top genres based on the number of movies?
- Which genres have the highest popularity on average?
- Do certain genres receive more votes or higher ratings?

What are the top genres based on the number of movies

In [35]:
```
# Check the first few rows to understand the genre format
df['Genre'].head()
```

Out[35]:
```
0    Action, Adventure, Science Fiction
1             Crime, Mystery, Thriller
2                              Thriller
3    Animation, Comedy, Family, Fantasy
4      Action, Adventure, Thriller, War
Name: Genre, dtype: object
```

Observations:-

- Movies can have multiple genres, separated by commas.
- Some movies belong to a single genre (e.g., "Thriller"), while others have multiple classifications (e.g., "Action, Adventure, Science Fiction").
- This format requires splitting genres for deeper analysis (e.g., most common genres, genre-based trends).

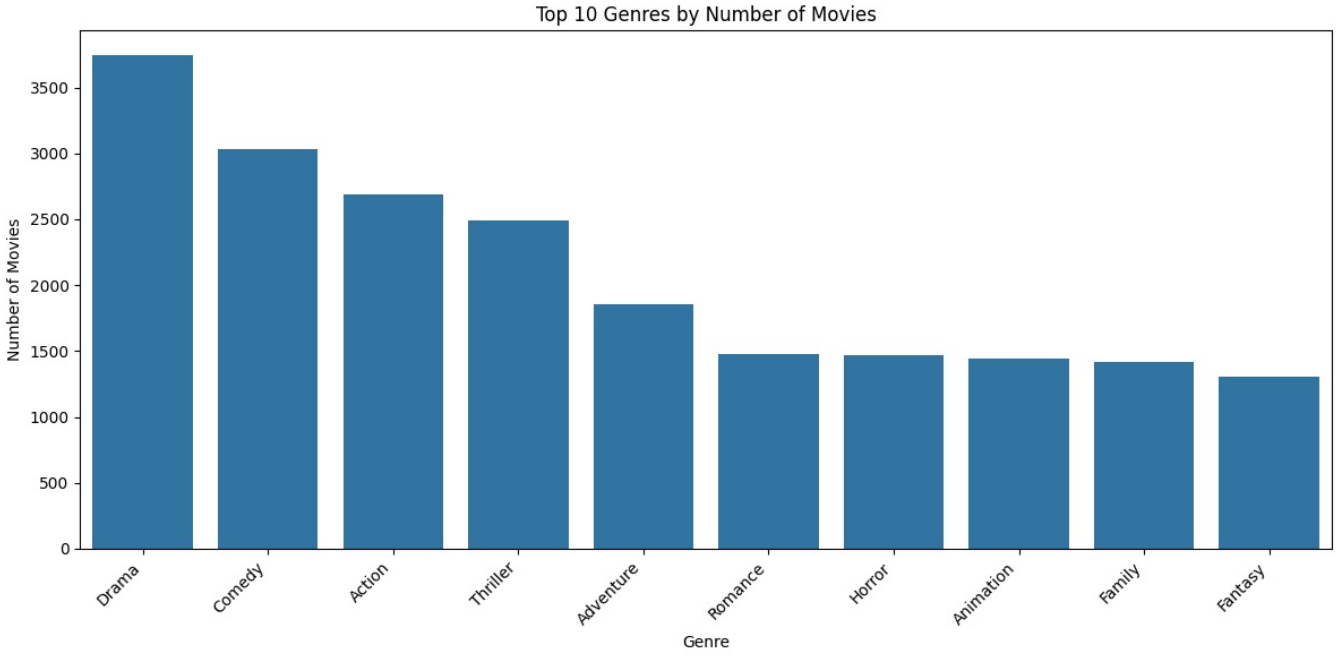
In [36]:
```
# Since genres are stored as comma-separated strings, we need to split them
# Create a list of all genres
all_genres = []
for genres in df['Genre'].dropna():
    genre_list = [g.strip() for g in genres.split(',')]
    all_genres.extend(genre_list)

# Count the frequency of each genre
genre_counts = pd.Series(all_genres).value_counts()

print("Top genres based on number of movies:-")
print(genre_counts.head(10))
```

```
Top genres based on number of movies:-
Drama        3744
Comedy       3031
Action       2686
Thriller     2488
Adventure    1853
Romance      1476
Horror       1470
Animation    1439
Family       1414
Fantasy      1308
Name: count, dtype: int64
```

Observations:-

- Drama (3,744) is the most common genre, followed by Comedy (3,031) and Action (2,686).
- Thriller (2,488) and Adventure (1,853) are also popular categories.
- Fantasy (1,308) and Family (1,414) genres are less common compared to others.
- The dataset includes a mix of serious (Drama, Thriller) and entertaining (Comedy, Animation) genres.

In [37]:
```
# Visualize the top genres
plt.figure(figsize=(12, 6))
sns.barplot(x=genre_counts.head(10).index, y=genre_counts.head(10).values)
plt.title('Top 10 Genres by Number of Movies')
plt.xlabel('Genre')
plt.ylabel('Number of Movies')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```



Observation:-

- Drama is the most common genre, followed by Comedy and Action.
- Thriller and Adventure also have a significant number of movies.
- Fantasy, Family, and Animation are the least frequent among the top 10 genres.
- The distribution shows a mix of serious (Drama, Thriller) and entertaining (Comedy, Adventure) genres.

Observation genre-based trends in:-

- Average Popularity per Genre (Which genres attract the most attention?)
- Average Vote Count per Genre (Which genres get the most votes?)
- Average Vote Rating per Genre (Which genres are rated the highest?)

Which genres have the highest popularity on average

In [38]:
```
# Create a dictionary to store popularity sums and counts for each genre
genre_popularity = {}
```

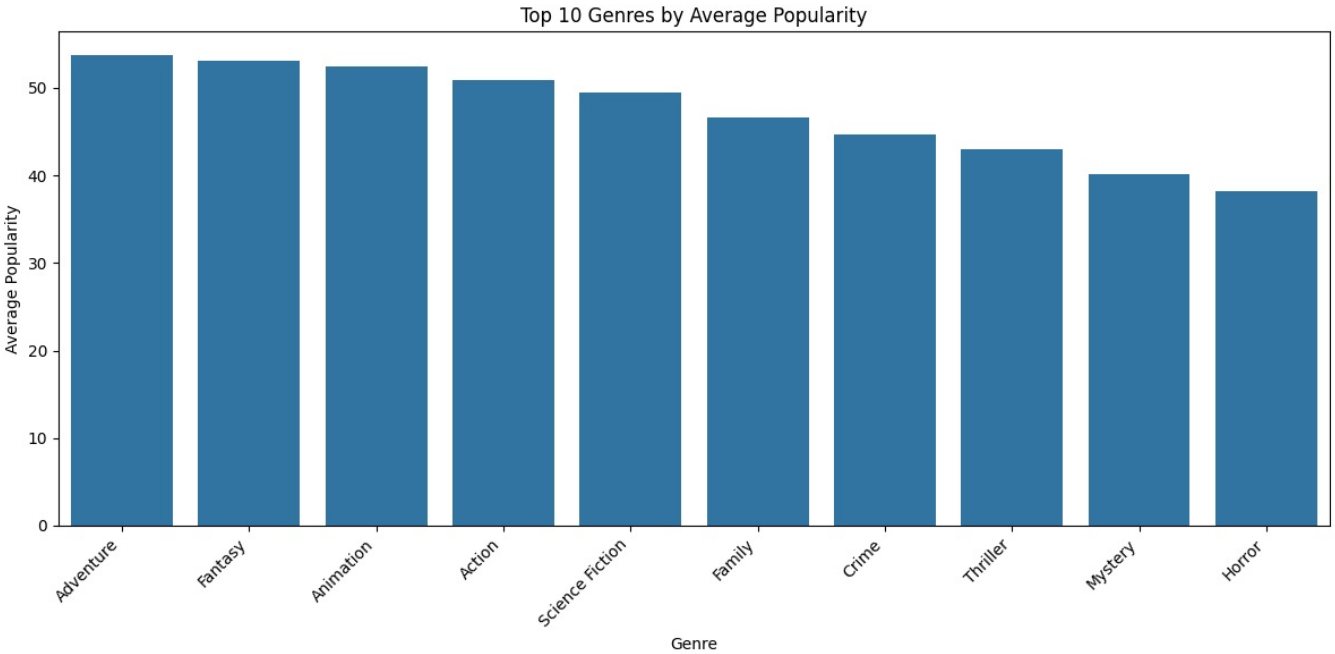
```
genre_counts_dict = {}

for i, row in df.iterrows():
    if pd.notna(row['Genre']):
        genres = [g.strip() for g in row['Genre'].split(',')]
        for genre in genres:
            if genre not in genre_popularity:
                genre_popularity[genre] = 0
                genre_counts_dict[genre] = 0
            genre_popularity[genre] += row['Popularity']
            genre_counts_dict[genre] += 1

# Calculate average popularity for each genre
avg_popularity = {genre: genre_popularity[genre] / genre_counts_dict[genre]
                  for genre in genre_popularity}

# Convert to Series and sort
avg_popularity_series = pd.Series(avg_popularity).sort_values(ascending=False)

print("Genres with highest average popularity:-")
print(avg_popularity_series.head(10))
```

```
Genres with highest average popularity:-
Adventure          53.742888
Fantasy            53.081342
Animation          52.433848
Action             50.890133
Science Fiction    49.511334
Family             46.610616
Crime              44.747295
Thriller           42.946258
Mystery            40.093590
Horror             38.264498
dtype: float64
```

Observations:-

- Adventure, Fantasy, and Animation have the highest average popularity.
- Action and Science Fiction are also among the most popular genres.
- Horror and Mystery have relatively lower popularity compared to other top genres.
- Family movies have good popularity, likely due to a broad audience appeal.

In [39]:
```
# Visualize average popularity by genre
plt.figure(figsize=(12, 6))
sns.barplot(x=avg_popularity_series.head(10).index, y=avg_popularity_series.head(10).values)
plt.title('Top 10 Genres by Average Popularity')
plt.xlabel('Genre')
plt.ylabel('Average Popularity')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```



Observations:-

- Adventure, Fantasy, and Animation are the most popular genres on average.

- Action and Science Fiction also have high popularity.
- Family movies are relatively popular, indicating broad appeal.
- Horror and Mystery have lower average popularity compared to other top genres.

Do certain genres receive more votes or higher ratings:-

In [40]:
```
# Calculate average vote count and vote average for each genre
genre_vote_count = {}
genre_vote_avg = {}
genre_counts_for_votes = {}

for i, row in df.iterrows():
    if pd.notna(row['Genre']):
        genres = [g.strip() for g in row['Genre'].split(',')]
        for genre in genres:
            if genre not in genre_vote_count:
                genre_vote_count[genre] = 0
                genre_vote_avg[genre] = 0
                genre_counts_for_votes[genre] = 0
            genre_vote_count[genre] += row['Vote_Count']
            genre_vote_avg[genre] += row['Vote_Average']
            genre_counts_for_votes[genre] += 1

# Calculate averages
avg_vote_count = {genre: genre_vote_count[genre] / genre_counts_for_votes[genre]
                  for genre in genre_vote_count}
avg_vote_rating = {genre: genre_vote_avg[genre] / genre_counts_for_votes[genre]
                   for genre in genre_vote_avg}
```

In [41]:
```
# Convert to Series and sort
avg_vote_count_series = pd.Series(avg_vote_count).sort_values(ascending=False)
avg_vote_count_series.head(10)
```

Out[41]:
```
Adventure          2328.045872
Science Fiction    2239.179890
Fantasy            1928.088685
Action             1812.611690
Crime              1594.413043
Mystery            1528.169470
Thriller           1459.643891
War                1457.525974
Family             1447.400990
Drama              1373.262553
dtype: float64
```

Observations:-

- Adventure, Science Fiction, and Fantasy have the highest average vote counts, indicating strong audience engagement.
- Action and Crime genres also receive high votes, suggesting popularity among viewers.
- Thriller, War, and Family movies have moderate vote counts.
- Drama, despite being the most common genre, has relatively lower votes on average.

In [42]:
```
avg_vote_rating_series = pd.Series(avg_vote_rating).sort_values(ascending=False)
avg_vote_rating_series.head(10)
```

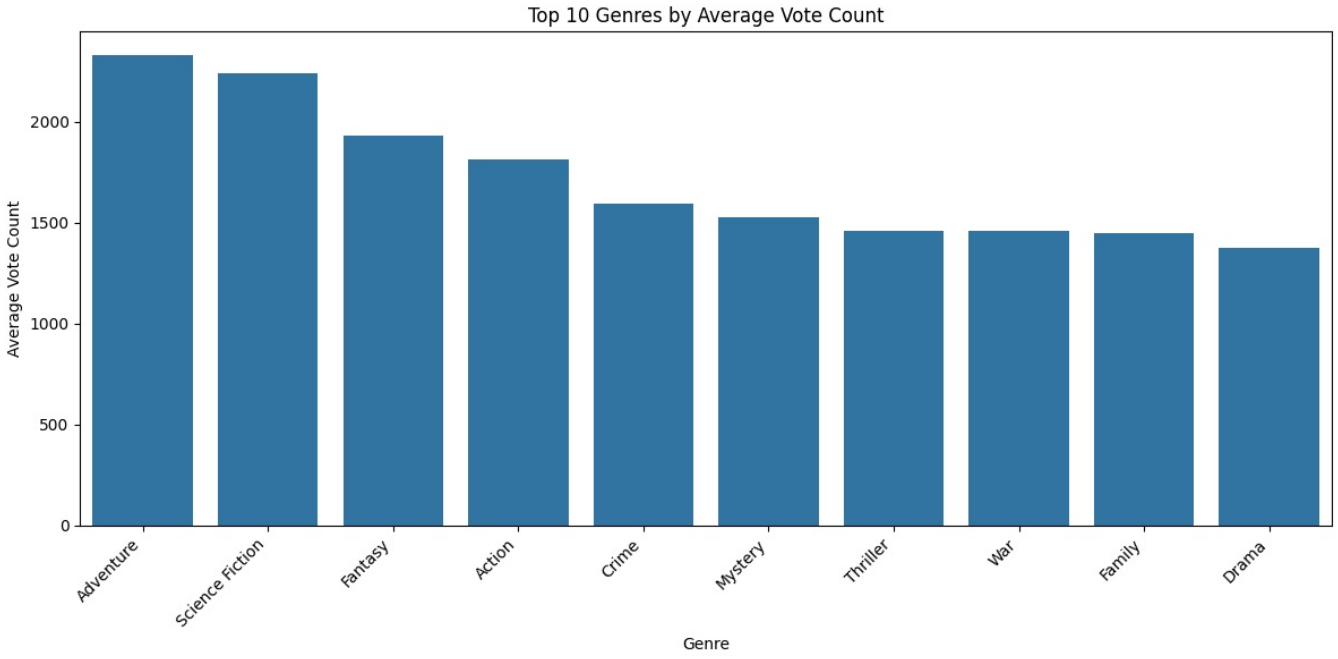
Out[42]:
```
History        6.965574
War            6.948701
Music          6.879322
Animation      6.846560
Western        6.754745
Drama          6.706143
Documentary    6.663721
Family         6.581047
Romance        6.560772
Crime          6.552013
dtype: float64
```

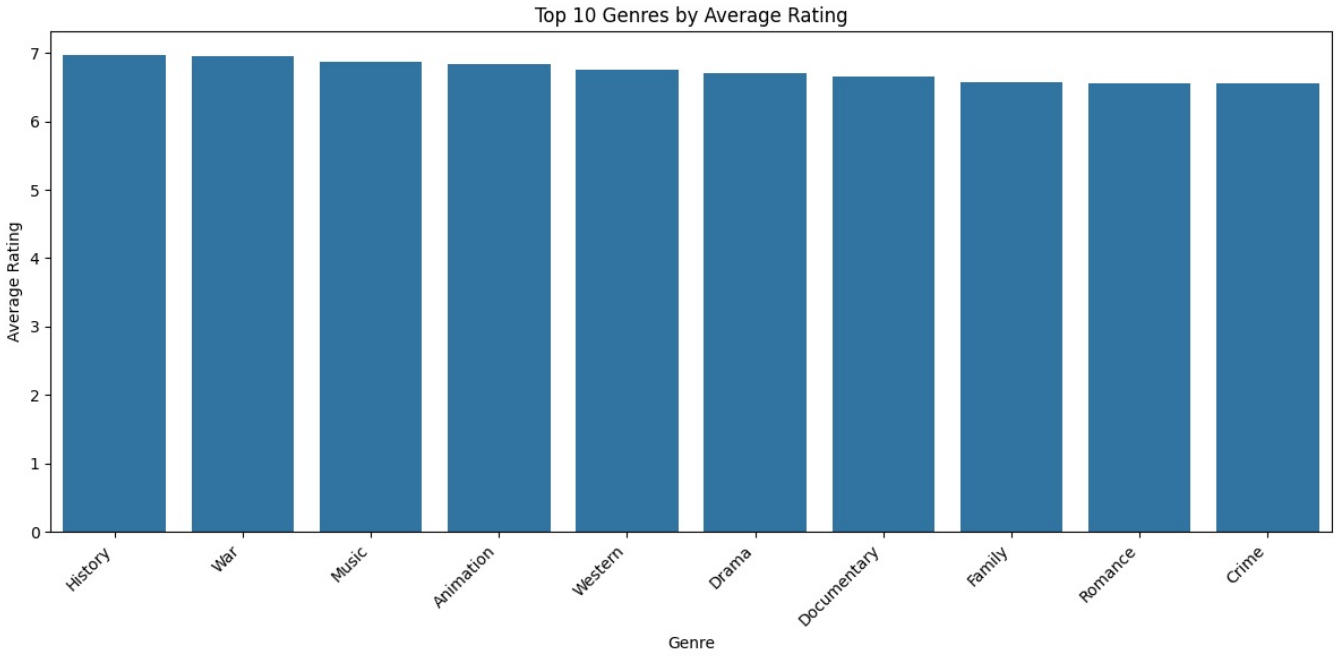
Observations:-

- History, War, and Music genres have the highest average ratings, indicating strong audience appreciation.
- Animation and Western also receive high ratings, showing their niche appeal.
- Drama, Documentary, and Family genres maintain good ratings, reflecting their emotional and storytelling depth.
- Romance and Crime also have decent ratings, suggesting consistent audience engagement.

In [43]:
```
# Visualize average vote count by genre
plt.figure(figsize=(12, 6))
sns.barplot(x=avg_vote_count_series.head(10).index, y=avg_vote_count_series.head(10).values)
```

```
plt.title('Top 10 Genres by Average Vote Count')
plt.xlabel('Genre')
plt.ylabel('Average Vote Count')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()

# Visualize average rating by genre
plt.figure(figsize=(12, 6))
sns.barplot(x=avg_vote_rating_series.head(10).index, y=avg_vote_rating_series.head(10).values)
plt.title('Top 10 Genres by Average Rating')
plt.xlabel('Genre')
plt.ylabel('Average Rating')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```





Observations:-

Top Vote Count Genres

- Adventure and Science Fiction get the highest audience votes, showing greater audience engagement.
- Drama and Family genres receive fewer votes despite being common.

Top Average Rating Genres

- History, War, and Music genres have the highest ratings, indicating quality content.
- Crime and Romance have moderate ratings but higher audience engagement.

In [44]:
```
# Create a combined visualization for comparison
# Select top 10 genres by movie count for comparison
top_genres = genre_counts.head(10).index

# Create a dataframe for these genres with all metrics
comparison_data = pd.DataFrame({
    'Genre': top_genres,
    'Movie Count': [genre_counts[genre] for genre in top_genres],
    'Avg Popularity': [avg_popularity.get(genre, 0) for genre in top_genres],
    'Avg Vote Count': [avg_vote_count.get(genre, 0) for genre in top_genres],
    'Avg Rating': [avg_vote_rating.get(genre, 0) for genre in top_genres]
})

print("Comparison of metrics for top 10 genres by movie count:-")
print(comparison_data)
```

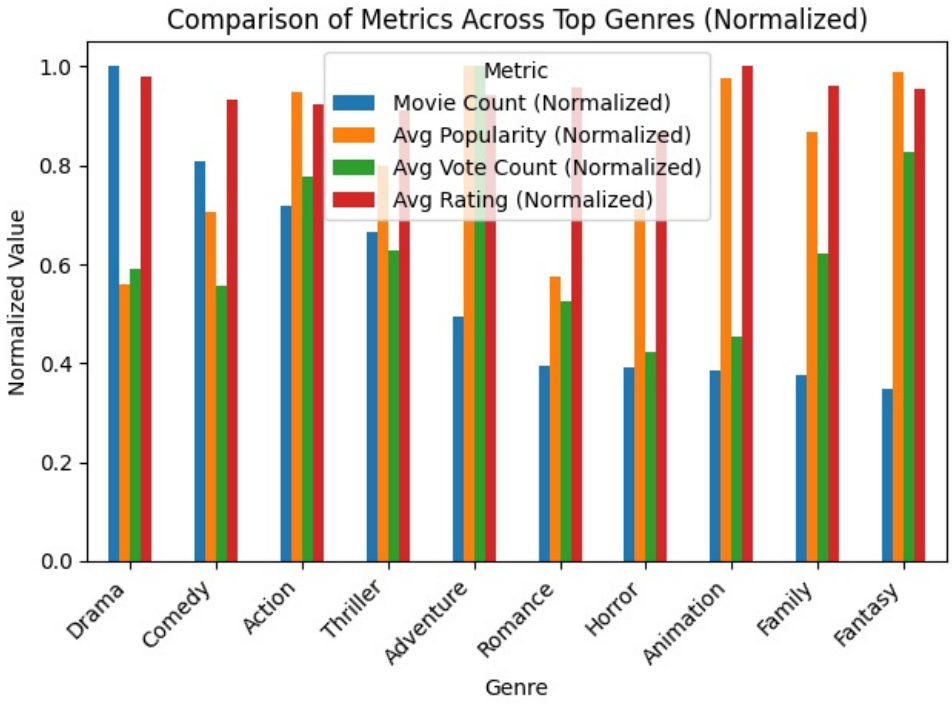
```
Comparison of metrics for top 10 genres by movie count:-
       Genre  Movie Count  Avg Popularity  Avg Vote Count  Avg Rating
0      Drama         3744       30.077651     1373.262553    6.706143
1     Comedy         3031       37.873669     1297.755526    6.382976
2     Action         2686       50.890133     1812.611690    6.330268
3   Thriller         2488       42.946258     1459.643891    6.246141
4  Adventure         1853       53.742888     2328.045872    6.456719
5    Romance         1476       30.866030     1222.056911    6.560772
6     Horror         1470       38.264498      985.364626    5.940068
7  Animation         1439       52.433848     1059.794997    6.846560
8     Family         1414       46.610616     1447.400990    6.581047
9    Fantasy         1308       53.081342     1928.088685    6.526300
```

Observations:-

- Adventure, Fantasy, and Animation have high popularity and good vote counts, making them audience favorites.
- Drama has the highest movie count but lower popularity than Action and Adventure.
- Horror has a low average rating (5.94), suggesting mixed audience reception.
- History and Animation maintain high average ratings, indicating consistent quality.

In [45]:
```
# Normalize the data for better visualization
for col in ['Movie Count', 'Avg Popularity', 'Avg Vote Count', 'Avg Rating']:
    max_val = comparison_data[col].max()
    comparison_data[f'{col} (Normalized)'] = comparison_data[col] / max_val

# Plot the normalized metrics for comparison
plt.figure(figsize=(14, 8))
comparison_data.set_index('Genre')[['Movie Count (Normalized)',
                                    'Avg Popularity (Normalized)',
                                    'Avg Vote Count (Normalized)',
                                    'Avg Rating (Normalized)']].plot(kind='bar')
plt.title('Comparison of Metrics Across Top Genres (Normalized)')
plt.xlabel('Genre')
plt.ylabel('Normalized Value')
plt.legend(title='Metric')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```

```
<Figure size 1400x800 with 0 Axes>
```



Observations:-

- Drama has the highest movie count but lower popularity and votes.
- Adventure, Fantasy, and Animation score high in popularity, votes, and ratings, making them audience favorites.
- Horror has lower ratings despite decent popularity.
- Romance has moderate popularity and votes but a higher rating than some other genres.

3)Release Date & Trends

- How many movies are released each year?
- What are the trends in movie ratings over time?
- Does popularity or vote count change over the years?

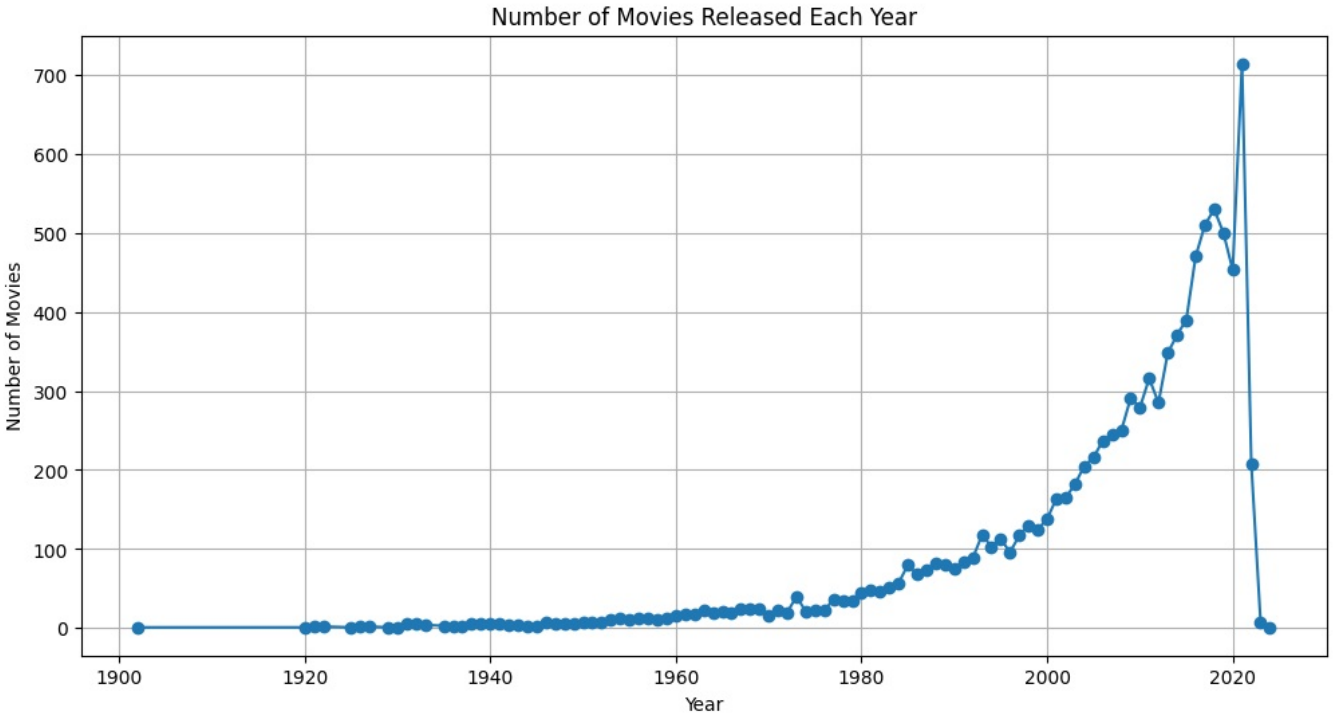
In [46]:
```
# Convert Release_Date to datetime and extract year
df['Release_Date'] = pd.to_datetime(df['Release_Date'], errors='coerce')
df['Year'] = df['Release_Date'].dt.year
```

How many movies are released each year:-

In [47]:
```
movies_per_year = df['Year'].value_counts().sort_index()
print("Movies released each year:-")
print(movies_per_year)
```

```
Movies released each year:-
Year
1902      1
1920      1
1921      2
1922      2
1925      1
        ...
2020    453
2021    714
2022    208
2023      8
2024      1
Name: count, Length: 102, dtype: int64
```

In [48]:
```python
# Visualization how many number of movies released in each year
plt.figure(figsize=(12,6))
plt.plot(movies_per_year.index, movies_per_year.values, marker='o')
plt.title('Number of Movies Released Each Year')
plt.xlabel('Year')
plt.ylabel('Number of Movies')
plt.grid(True)
plt.show()
```



Observations:-

- The number of movies released increased steadily from the 1900s, with a sharp rise after 2000.
- A peak around 2018-2019 suggests a boom in movie production.
- A sudden drop after 2020 likely reflects the impact of the COVID-19 pandemic.
- The overall trend indicates strong industry growth over time.

What are the trends in movie ratings over time:-

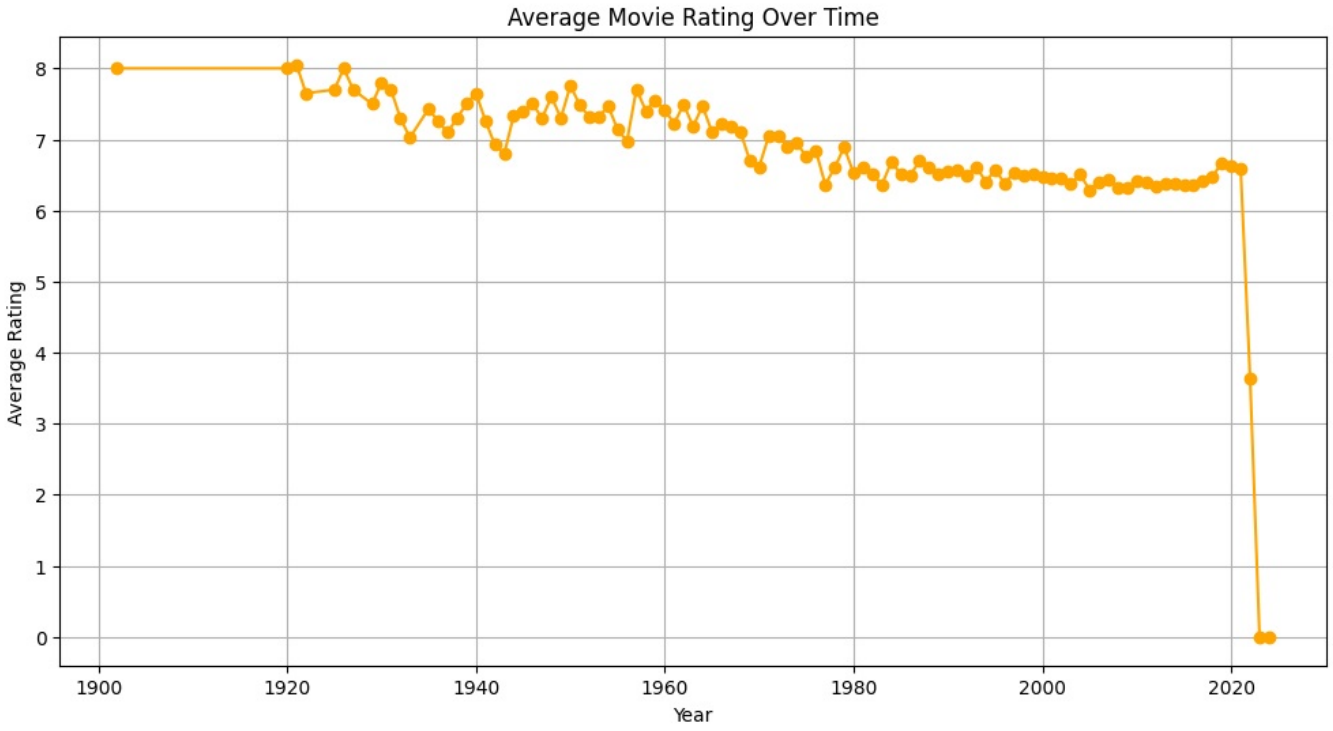
In [49]:
```python
ratings_per_year = df.groupby('Year')['Vote_Average'].mean()
print("Average movie rating per year:-")
print(ratings_per_year)
```

```
Average movie rating per year:-
Year
1902    8.000000
1920    8.000000
1921    8.050000
1922    7.650000
1925    7.700000
          ...
2020    6.621413
2021    6.584734
2022    3.635577
2023    0.000000
2024    0.000000
Name: Vote_Average, Length: 102, dtype: float64
```

Observations:-

- Early movies (1902-1925) had high average ratings (above 7.5).
- Ratings remained relatively stable over time, mostly around 6-7 in recent decades.
- 2022 shows a sharp drop in average rating (~3.63), possibly due to fewer votes or lower-quality releases.
- 2023 and 2024 have 0 ratings, likely due to missing or unvoted movies.

In [50]:
```
# Visualization
plt.figure(figsize=(12,6))
plt.plot(ratings_per_year.index, ratings_per_year.values, marker='o', color='orange')
plt.title('Average Movie Rating Over Time')
plt.xlabel('Year')
plt.ylabel('Average Rating')
plt.grid(True)
plt.show()
```



Observations:-

- Early movies (1900-1930s) had higher ratings (~8).
- From 1940s onward, ratings stabilized around 6-7.
- Recent years (2022-2024) show a sharp drop, likely due to fewer ratings or missing data.
- The sudden drop to 0 in 2023-2024 suggests incomplete data rather than a true rating decline.

Does popularity or vote count change over the years:-

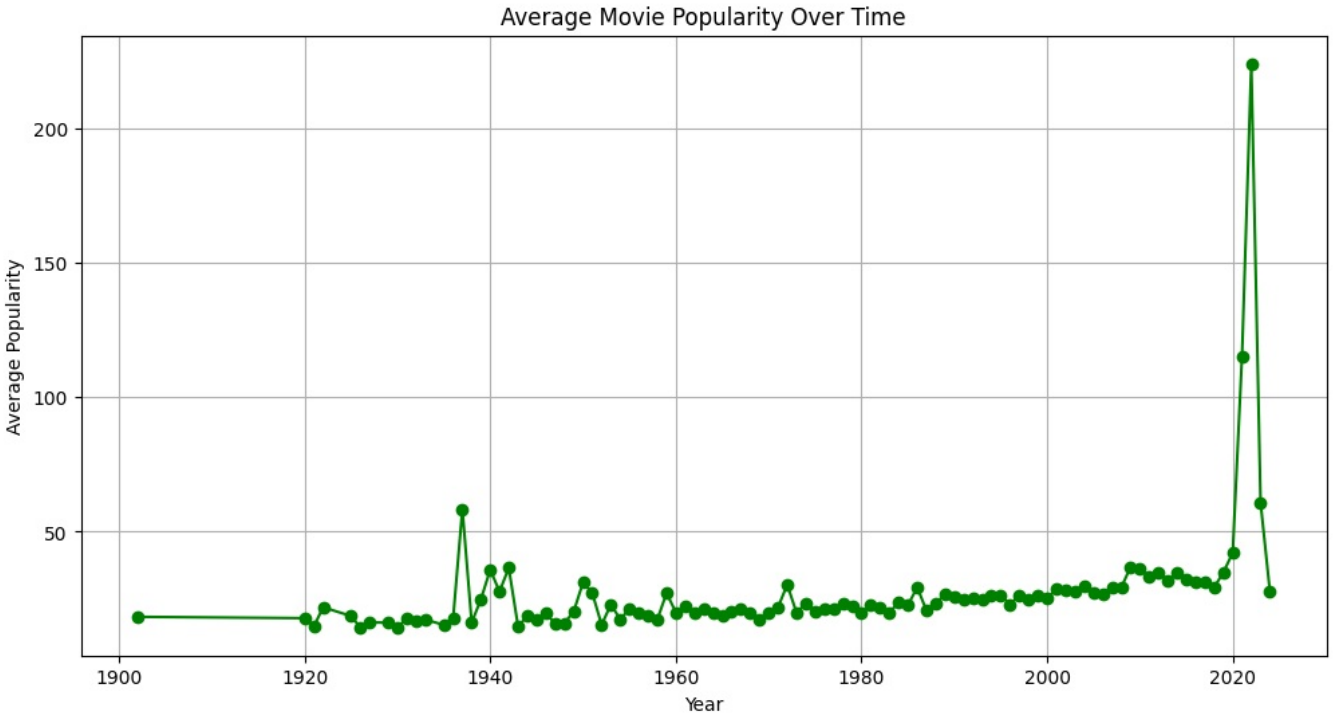
In [51]:
```
# Average popularity per year
avg_popularity_per_year = df.groupby('Year')['Popularity'].mean()
print("Average popularity per year:")
print(avg_popularity_per_year)
```

```
Average popularity per year:
Year
1902     18.356000
1920     17.858000
1921     14.854500
1922     21.901000
1925     18.663000
           ...
2020     42.191951
2021    115.252629
2022    223.867346
2023     60.779125
2024     27.987000
Name: Popularity, Length: 102, dtype: float64
```

Observations:-

- Older movies (1900-1950s) had low popularity (~15-22).
- Popularity steadily increased over time, peaking in 2022 (223.86).
- 2023-2024 show a decline, possibly due to fewer ratings or data gaps.
- The spike in 2021-2022 suggests a surge in movie interest, maybe due to streaming growth.

In [52]:
```
# Visualization:-
plt.figure(figsize=(12,6))
plt.plot(avg_popularity_per_year.index, avg_popularity_per_year.values, marker='o', color='green')
plt.title('Average Movie Popularity Over Time')
plt.xlabel('Year')
plt.ylabel('Average Popularity')
plt.grid(True)
plt.show()
```



Observations:-

- Steady low popularity from 1900 to early 2000s.
- Small spikes in the 1940s and 1980s, possibly due to iconic films.
- Sharp surge after 2020, peaking in 2022, likely due to streaming platforms.
- Drop in 2023-2024, indicating reduced audience engagement or data gaps.

In [53]:
```
# Average vote count per year
avg_vote_count_per_year = df.groupby('Year')['Vote_Count'].mean()
print("Average vote count per year:")
print(avg_vote_count_per_year)
```

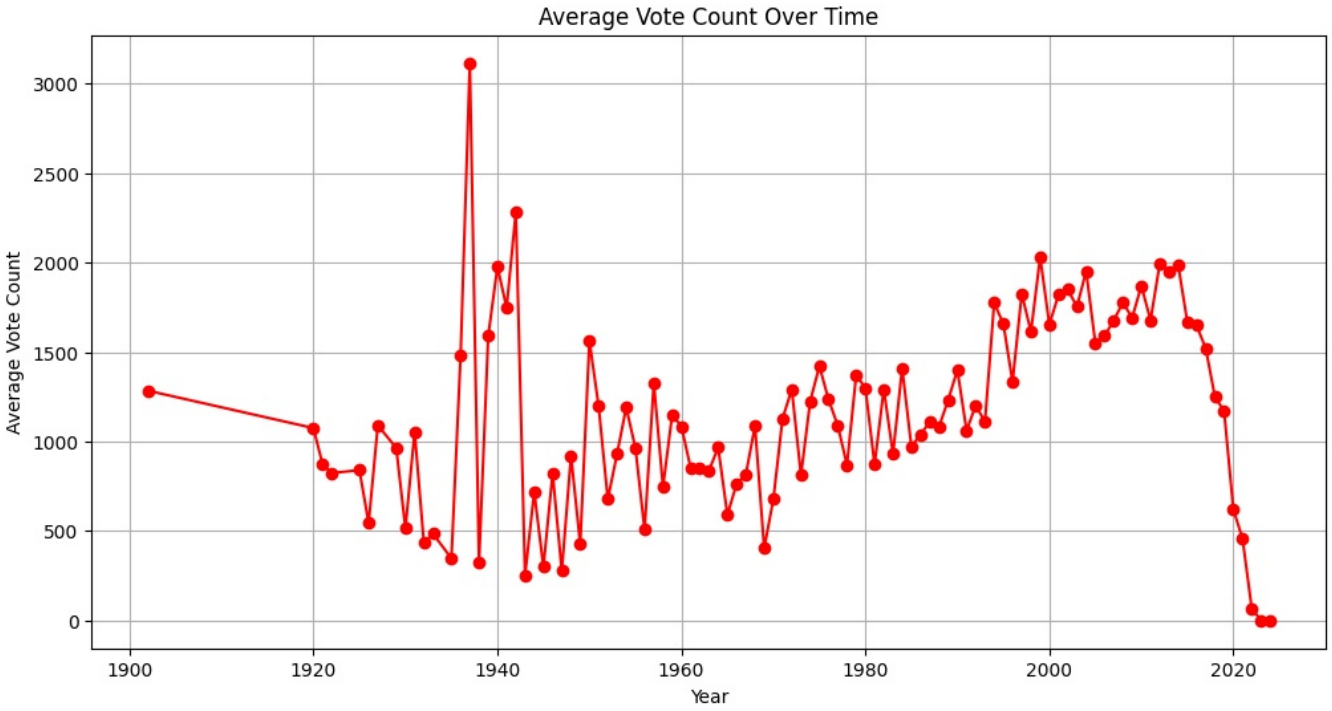
```
Average vote count per year:
Year
1902    1284.000000
1920    1075.000000
1921     870.500000
1922     823.500000
1925     840.000000
           ...
2020     620.642384
2021     456.612045
2022      67.841346
2023       0.000000
2024       0.000000
Name: Vote_Count, Length: 102, dtype: float64
```

Observations:-

- Higher vote counts in early years (1902–1925).
- Gradual decline over time, with fluctuations.
- Sharp drop after 2020, with zero votes in 2023–2024, likely due to missing data or fewer releases.

In [54]:
```
# Visualization:-
plt.figure(figsize=(12,6))
```

```
plt.plot(avg_vote_count_per_year.index, avg_vote_count_per_year.values, marker='o', color='red')
plt.title('Average Vote Count Over Time')
plt.xlabel('Year')
plt.ylabel('Average Vote Count')
plt.grid(True)
plt.show()
```



Observations:-

- Fluctuations before 1960, with some sharp spikes (likely iconic films).
- Steady increase from 1970s to 2010s, indicating growing audience engagement.
- Significant drop after 2020, likely due to fewer movie releases or data issues.

4)Language Analysis:-

- What are the most common languages in the dataset?
- How do movies in different languages compare in terms of ratings and popularity?

What are the most common languages in the dataset

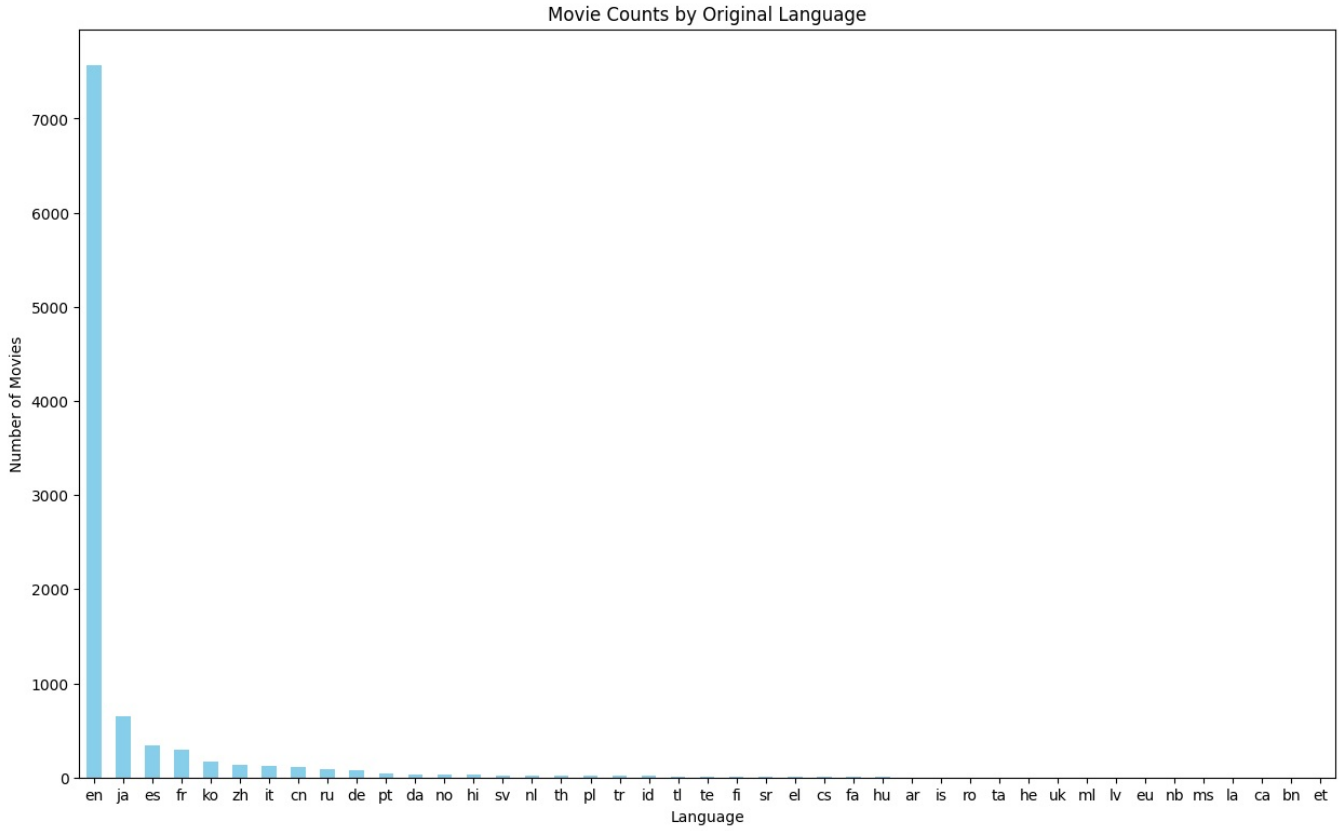
In [55]:
```
language_counts = df['Original_Language'].value_counts()
print("Most common languages:")
print(language_counts)
```

```
Most common languages:
Original_Language
en    7570
ja     645
es     339
fr     292
ko     170
zh     129
it     123
cn     109
ru      83
de      82
pt      37
da      28
no      26
hi      26
sv      23
nl      21
th      17
pl      17
tr      15
id      15
tl       8
te       6
fi       5
sr       5
el       5
cs       4
fa       3
hu       3
ar       2
is       2
ro       2
ta       2
he       2
uk       2
ml       1
lv       1
eu       1
nb       1
ms       1
la       1
ca       1
bn       1
et       1
Name: count, dtype: int64
```

Observations:-

- English (en) dominates with 7570 movies, far ahead of other languages.
- Japanese (ja), Spanish (es), and French (fr) follow, but with much lower counts.
- Many languages have very few movies (e.g., Latvian, Malay, Basque with only 1 each).

In [56]:
```python
# visualization:-
plt.figure(figsize=(15,9))
language_counts.plot(kind='bar', color='skyblue')
plt.title('Movie Counts by Original Language')
plt.xlabel('Language')
plt.ylabel('Number of Movies')
plt.xticks(rotation=0)
plt.show()
```



How do movies in different languages compare in terms of ratings and popularity

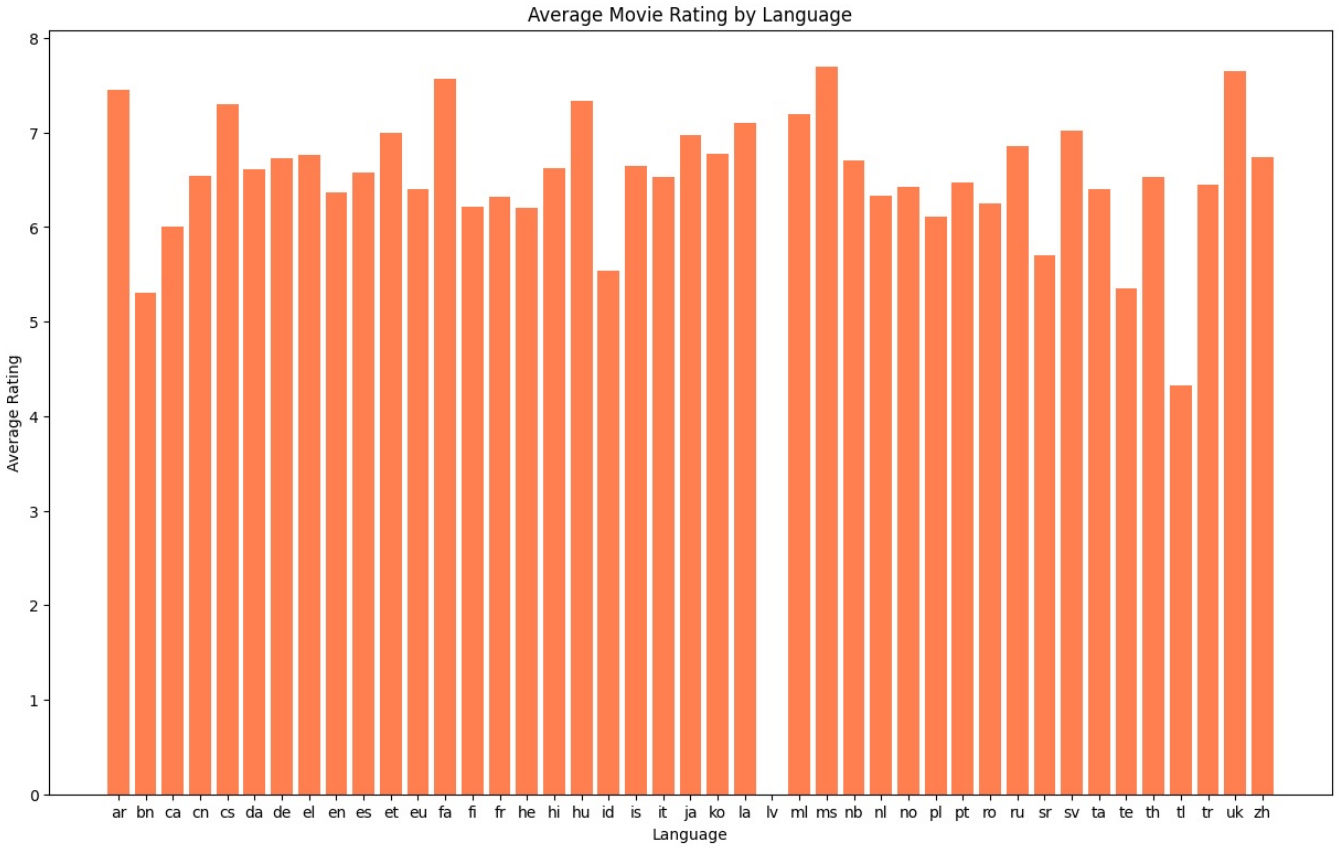
In [57]:
```
# Group by language and calculate average rating and average popularity
language_comparison = df.groupby('Original_Language').agg({'Vote_Average': 'mean', 'Popularity': 'mean', 'Vote_
print("Language comparison (Average Rating, Popularity, and Vote Count):")
print(language_comparison.head())
```

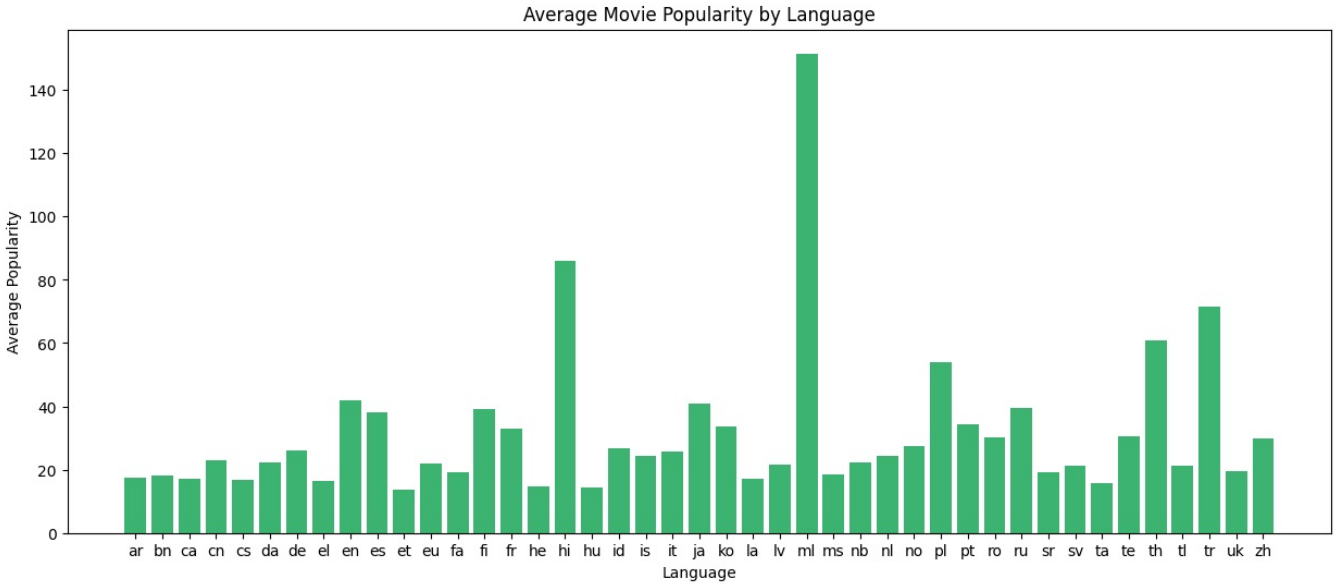
```
Language comparison (Average Rating, Popularity, and Vote Count):
  Original_Language  Vote_Average  Popularity  Vote_Count
0                ar      7.450000    17.29600  701.500000
1                bn      5.300000    18.05900   18.000000
2                ca      6.000000    17.21400   10.000000
3                cn      6.536697    23.14078  314.422018
4                cs      7.300000    16.70150  166.000000
```

Observations:-

- Arabic (ar) has the highest average rating (7.45) among the listed languages.
- Chinese (cn) movies are more popular (23.14 avg popularity) than others here.
- Vote count varies a lot, with Chinese (314 votes avg) and Catalan (ca) very low (10 votes avg).

In [58]:
```
# Plot average ratings by language
plt.figure(figsize=(15,9))
plt.bar(language_comparison['Original_Language'], language_comparison['Vote_Average'], color='coral')
plt.title('Average Movie Rating by Language')
plt.xlabel('Language')
plt.ylabel('Average Rating')
plt.xticks(rotation=0)
plt.show()

# Plot average popularity by language
plt.figure(figsize=(15,6))
plt.bar(language_comparison['Original_Language'], language_comparison['Popularity'], color='mediumseagreen')
plt.title('Average Movie Popularity by Language')
plt.xlabel('Language')
plt.ylabel('Average Popularity')
plt.xticks(rotation=0)
plt.show()
```





Observations:-

Top Graph (Average Ratings by Language)

- Most languages have ratings between 6 and 8.
- Some languages have higher average ratings, like ms (Malay) and lv (Latvian).
- Few languages have lower ratings than others.

Bottom Graph (Average Popularity by Language)

- Popularity varies widely across languages.
- Malay (ms) and Hungarian (hu) movies have the highest average popularity.
- Some languages have very low popularity despite decent ratings.

5)Relationship Between Popularity & Ratings:-

- Is there a correlation between popularity and ratings?

- Do highly rated movies tend to be more or less popular?

Is there a correlation between popularity and ratings

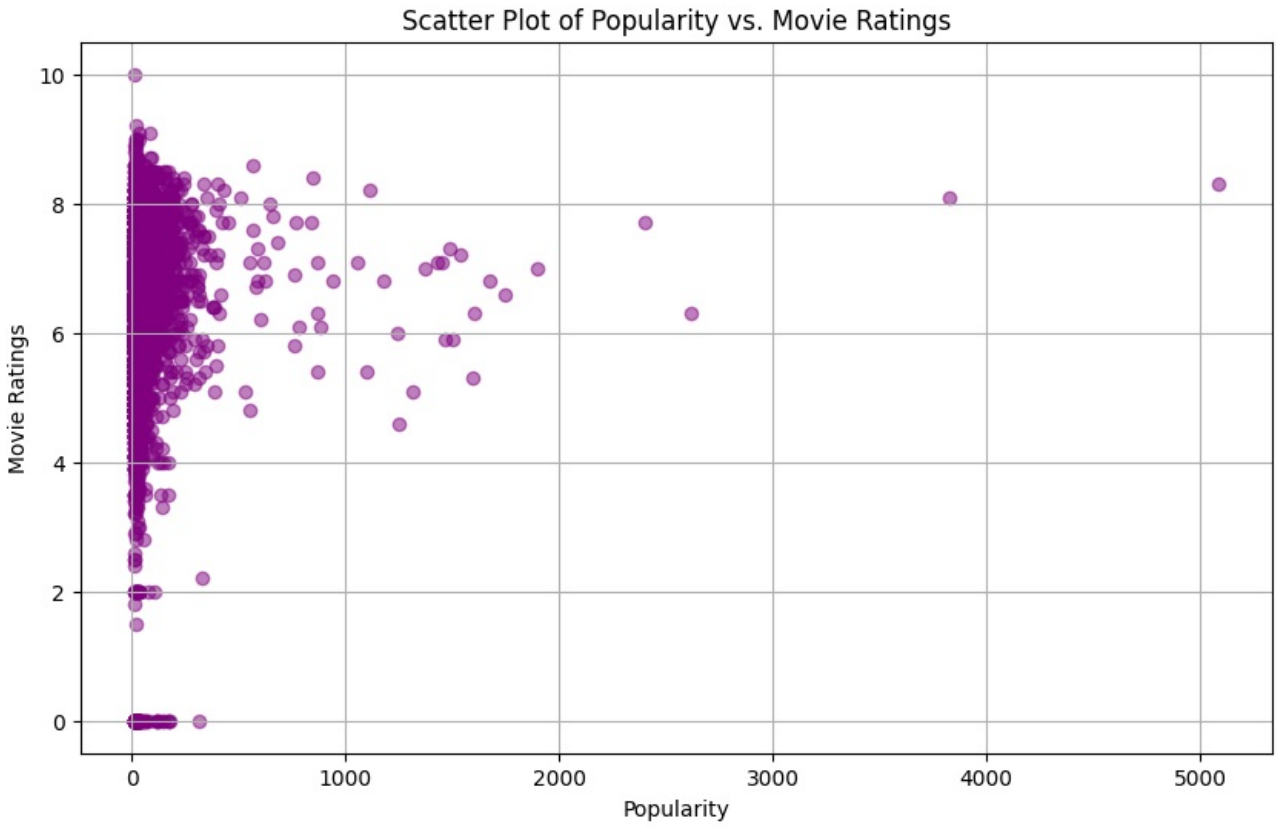
In [59]:
```
# Analyze correlation between popularity and ratings (Vote_Average)
correlation = df['Popularity'].corr(df['Vote_Average'])
print("Correlation between Popularity and Ratings:-", correlation)
```

```
Correlation between Popularity and Ratings:- 0.053843990775950194
```

Observation:-

- Weak correlation (0.0538) between popularity and ratings.
- Higher popularity does not strongly relate to better ratings.
- Popular movies aren't always highly rated and vice versa.

In [60]:
```
# Scatter plot between Popularity and Vote_Average
plt.figure(figsize=(10, 6))
plt.scatter(df['Popularity'], df['Vote_Average'], alpha=0.5, color='purple')
plt.title('Scatter Plot of Popularity vs. Movie Ratings')
plt.xlabel('Popularity')
plt.ylabel('Movie Ratings')
plt.grid(True)
plt.show()
```

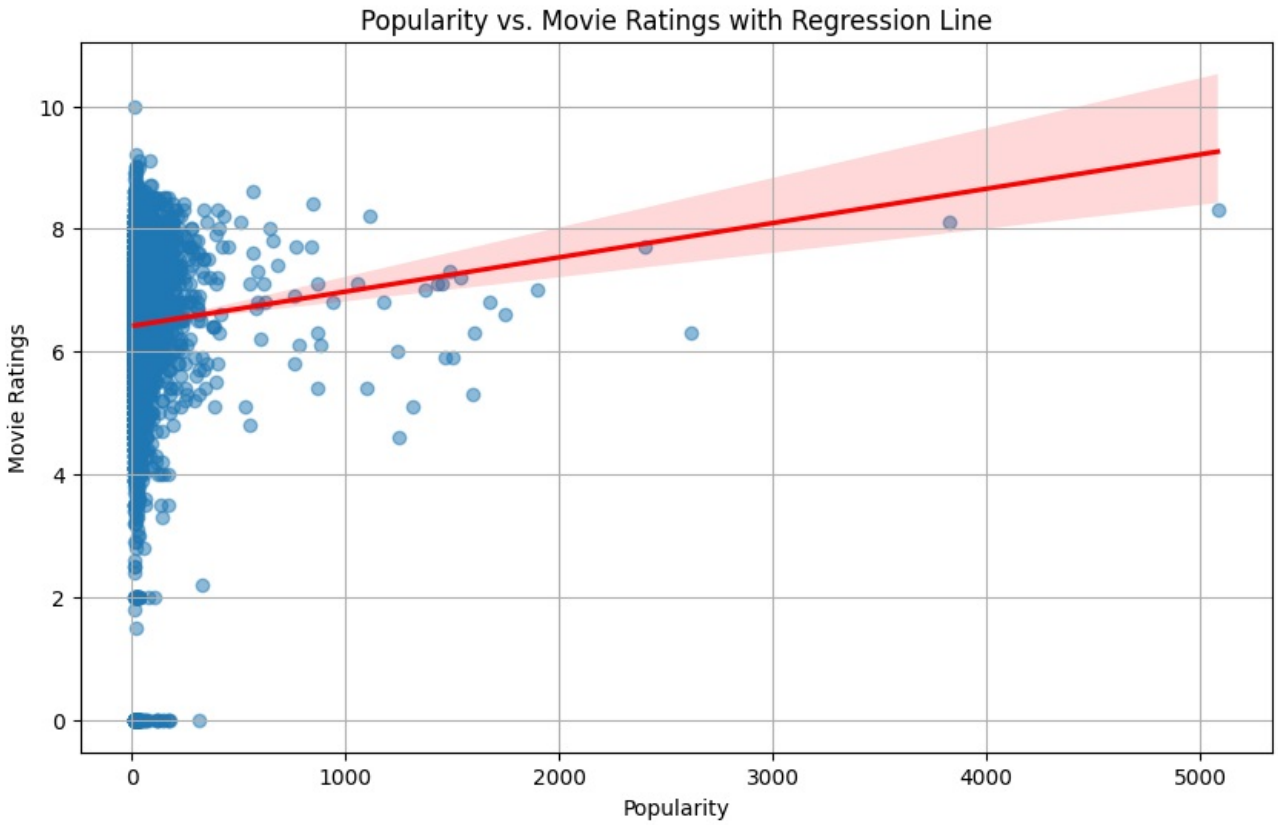


Observation:-

The scatter plot shows the distribution of movies based on their popularity and ratings. As you can see, the points are widely scattered without a clear pattern, confirming the weak correlation.

Do highly rated movies tend to be more or less popular

In [61]:
```
# To further illustrate, plot a regression line with seaborn
plt.figure(figsize=(10, 6))
sns.regplot(x='Popularity', y='Vote_Average', data=df, scatter_kws={'alpha':0.5}, line_kws={'color':'red'})
plt.title('Popularity vs. Movie Ratings with Regression Line')
plt.xlabel('Popularity')
plt.ylabel('Movie Ratings')
plt.grid(True)
plt.show()
```



Observation:-

The regression line (in red) is nearly horizontal, which further illustrates the weak relationship between popularity and ratings. This means that highly rated movies don't necessarily tend to be more popular, and popular movies don't necessarily have higher ratings.

Conclusion

Based on this analysis, we can conclude that:-

- There is a very weak correlation between popularity and ratings (correlation coefficient ≈ 0.054)
- Highly rated movies are not necessarily more popular
- A movie's popularity doesn't strongly predict its rating

This suggests that audience popularity and critical/user ratings are largely independent factors in this dataset.

In [ ]:

In [ ]:

In [ ]:

In [ ]: